# Zeladoria e Manutenção Urbana

## *Domain Driven Design Java*

Plataforma colaborativa para registrar danos, priorizar reparos e acompanhar a manutenção de espaços públicos.

Nome da equipe: GIT PUSH –FORCE

Nome da solução: CitizenTech: Zeladoria e Manutenção Urbana

### Integrantes


| Nome completo | RM |
|---|---|
| Bruno Lobosque | 561254 |
| Leonardo Vinicius de Souza | 562299 |
| Lucas Henrique Viana Estevam Sena | 566246 |


Repositório GitHub: https://github.com/1IMperaDOR0/citizentech

Deploy HTTPS: https://citizentech.onrender.com

# Sumário


| Seção | Página |
|---|---|
| 1 Especificação da solução | 3 |
| 2 Arquitetura e regras de negócio | 4 |
| 3 Diagramas de classes | 5 |
| 4 Protótipos de tela | 11 |
| 5 Contrato HTTP | 12 |
| 6 Execução e configuração | 14 |
| 7 Diagrama de entidades e relacionamentos | 15 |

# 1 Especificação da solução

Implementar o ciclo entre registro colaborativo de danos urbanos e manutenção, com rastreabilidade e prioridade explicável. A solução busca reduzir a dispersão de relatos e organizar a fila de trabalho pelo impacto sobre a mobilidade.

## Justificativa

Falhas em calçadas e vias afetam a segurança e o deslocamento, especialmente de cadeirantes, idosos e pessoas com carrinhos de bebê. Uma ocorrência centralizada permite reunir evidências de moradores e acompanhar o encaminhamento do reparo.

## Escopo

API RESTful em Spring Boot com persistência no Oracle FIAP. Contempla criação, consulta e edição de ocorrências; associação de relatos; sugestão de ocorrências próximas; priorização por regras; abertura de ordem de serviço; transições de estado; histórico e arquivamento lógico.

## Ferramentas e versões

| Componente | Configuração |
|---|---|
| Java | JDK 21; execução local informada com 21.0.11 |
| Spring Boot | 3.5.7 |
| Build | Maven 3.9.11 ou versão compatível |
| Persistência | JdbcTemplate com driver Oracle ojdbc11 |
| Banco | Oracle FIAP; H2 exclusivo dos testes automatizados |
| Desenvolvimento e publicação | IntelliJ IDEA, SQL Developer, Data Modeler, Postman, GitHub e Render com Docker |

# 2 Arquitetura e regras de negócio

| Camada | Responsabilidade |
|---|---|
| Controller | Rotas HTTP, DTOs validados, respostas e perfil autenticado. |
| BO | Regras, transações, autor dos relatos, transições e arquivamento. |
| DAO | SQL parametrizado, mapeamento JDBC, geração de chaves e bloqueio de linha. |
| Model | Ocorrencia, Relato, OrdemServico, HistoricoStatus, cadastros, StatusOcorrencia e Prioridade. |
| Exceções e configuração | ProblemDetail centralizado e perfis GESTOR e CIDADAO com Spring Security. |

## Regras centrais

Fluxo de estados: RECEBIDO → EM_ANALISE → EM_MANUTENCAO → CONCLUIDO. Não é permitido saltar etapas. Manutenção exige ordem de serviço. A criação grava ocorrência, primeiro relato e histórico em uma transação; a conclusão atualiza estado, histórico e data da ordem juntos.

Uma ocorrência admite no máximo uma OS. DELETE arquiva somente ocorrências RECEBIDO, preservando seus vínculos. Uma reincidência após conclusão deve ser registrada como nova ocorrência.

## Priorização e duplicidade

Pontuação: 50 por bloqueio total, 30 por proximidade de hospital e 20 por proximidade de escola. BAIXA abaixo de 20; MEDIA de 20 a 49; ALTA de 50 a 69; CRITICA a partir de 70. Candidatas abertas da mesma categoria a até 30 metros são sugeridas por Haversine, e o vínculo do relato é confirmado pelo usuário. Esses critérios são hipóteses do MVP, sem treinamento de IA.

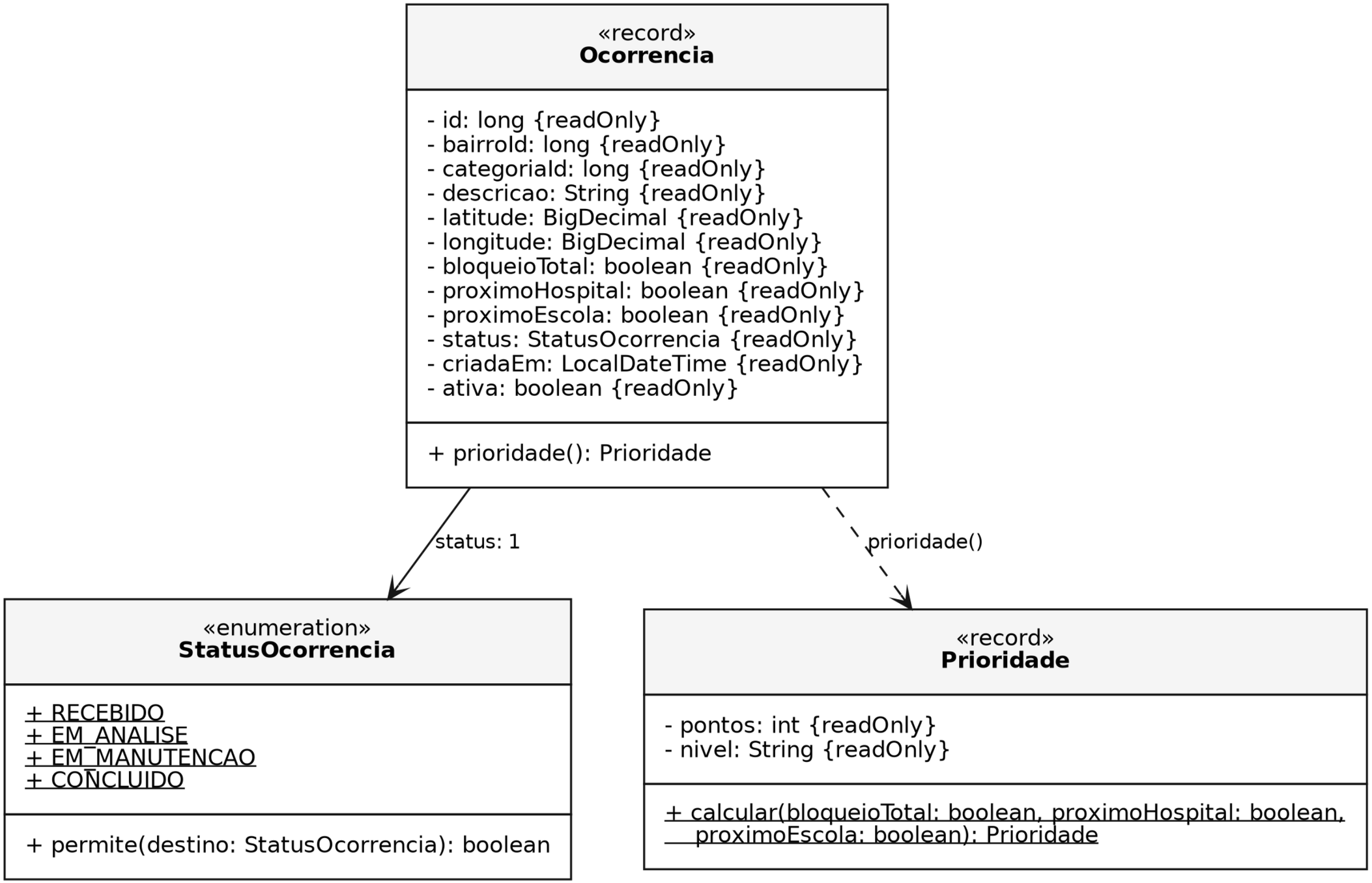
## Aplicação de DDD

A linguagem do domínio distingue ocorrência de relato. Estado e prioridade encapsulam regras específicas. A organização didática em DAO e BO atende ao escopo da disciplina; não pretende implementar todos os padrões táticos de DDD.

## 3 Diagramas de classes

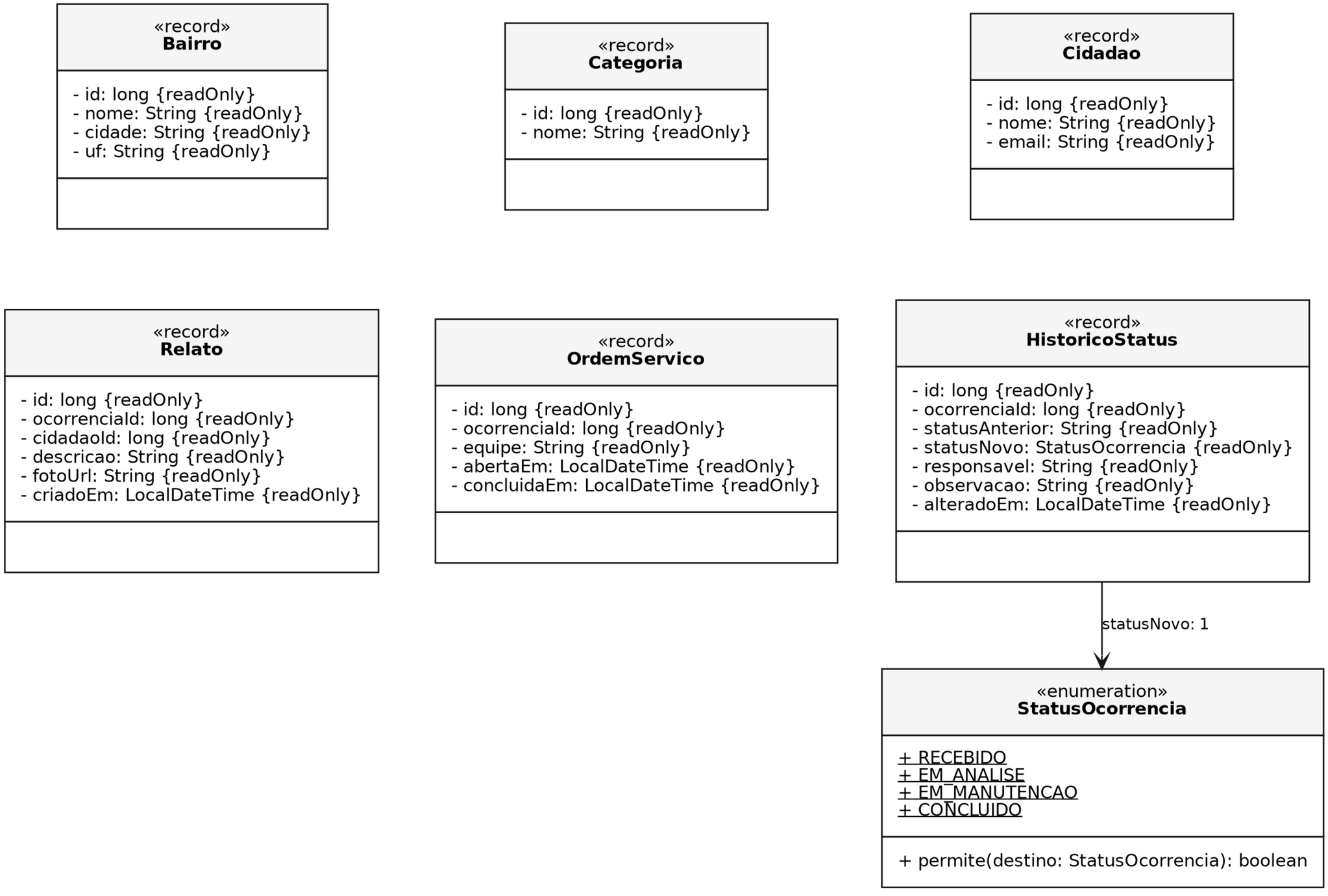
## Domínio da ocorrência

Ocorrência, estado e prioridade. Referências e tipos correspondem à implementação Java. Visibilidade: + público, - privado, # protegido e ~ pacote. Membros estáticos sublinhados. Nos records, acessores e construtores implícitos foram omitidos.



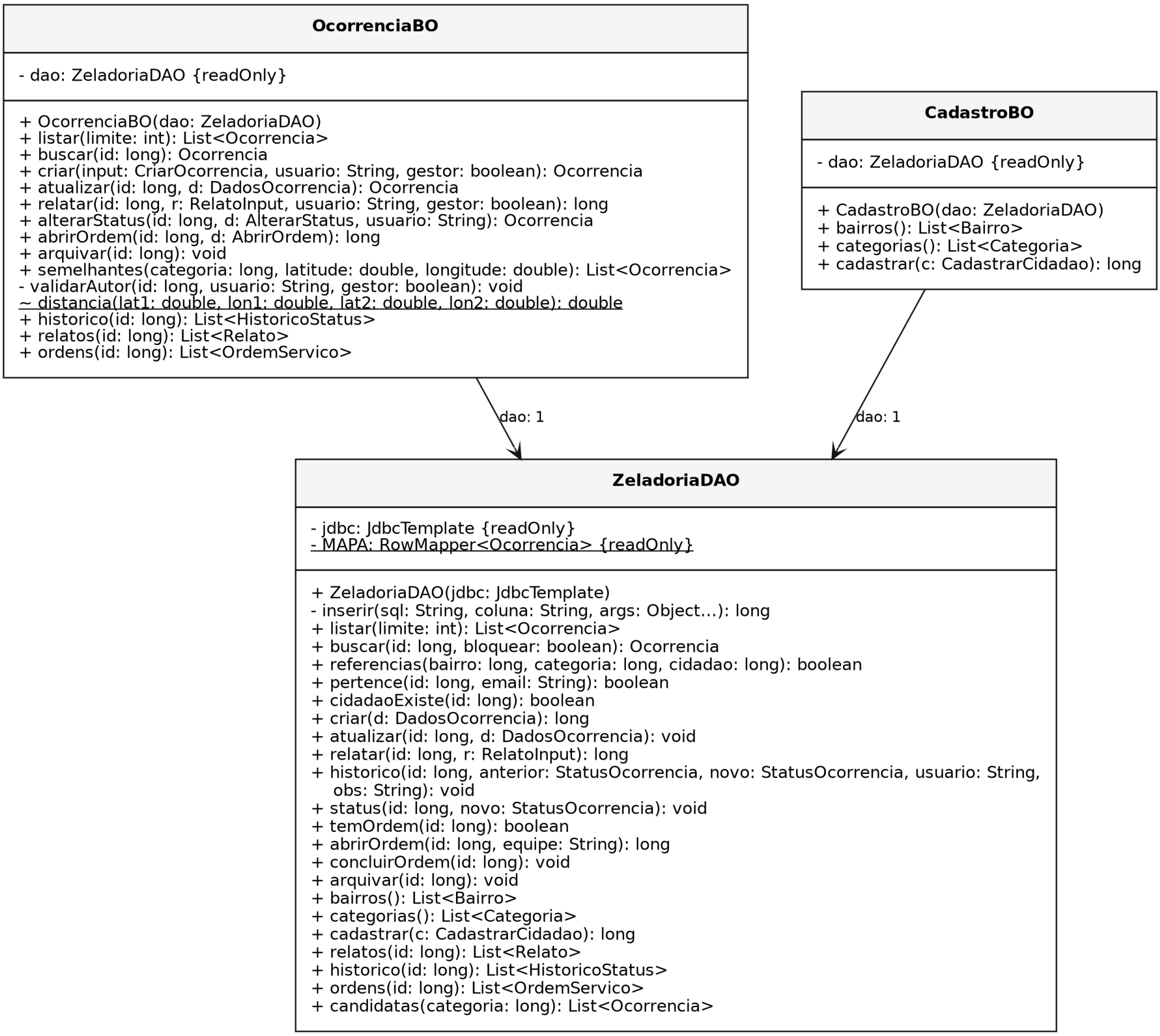
## Cadastros relatos e manutenção

Os vínculos persistidos são IDs numéricos, sem associações de objetos inventadas. As cardinalidades do banco aparecem no DER. Visibilidade: + público, - privado, # protegido e ~ pacote. Membros estáticos sublinhados. Nos records, acessores e construtores implícitos foram omitidos.



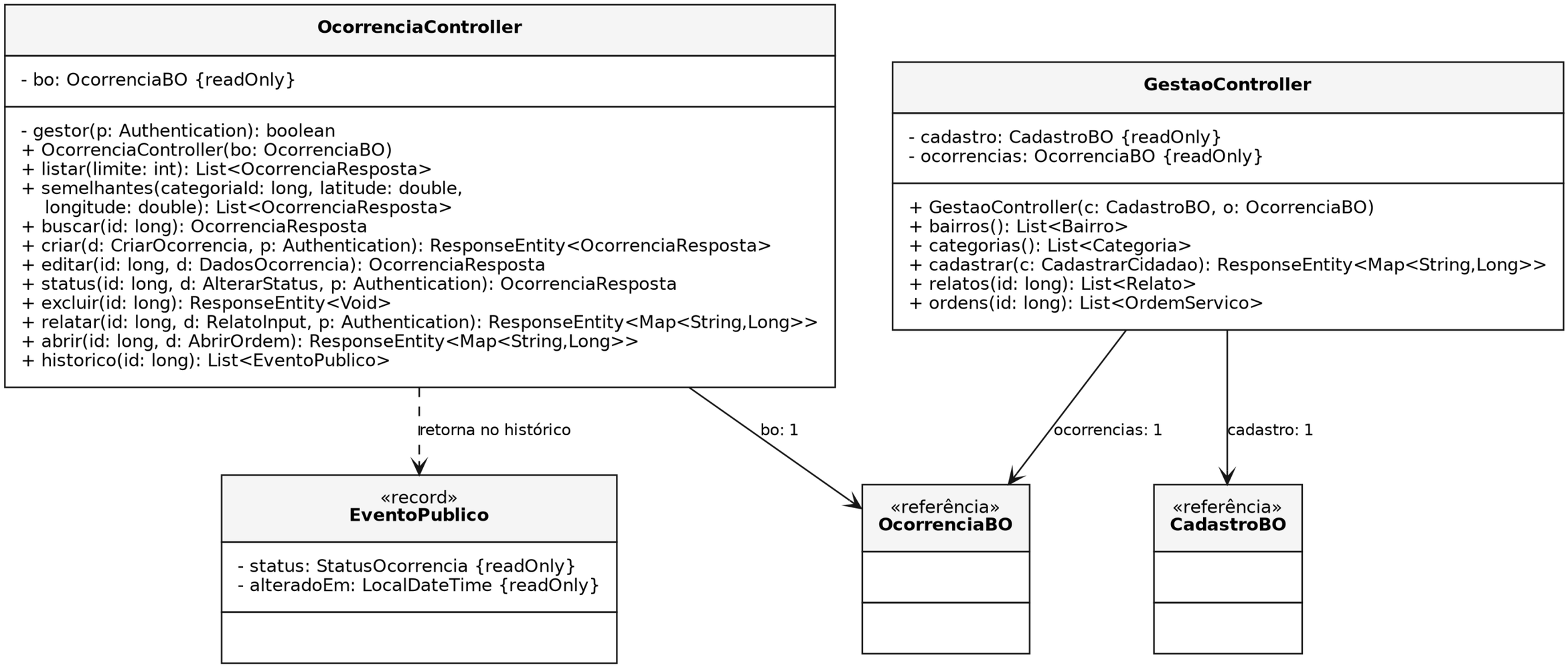
## Negócio e persistênca

BOs e DAO com atributos, métodos, parâmetros, retornos e visibilidades. Visibilidade: + público, - privado, # protegido e ~ pacote. Membros estáticos sublinhados. Nos records, acessores e construtores implícitos foram omitidos.



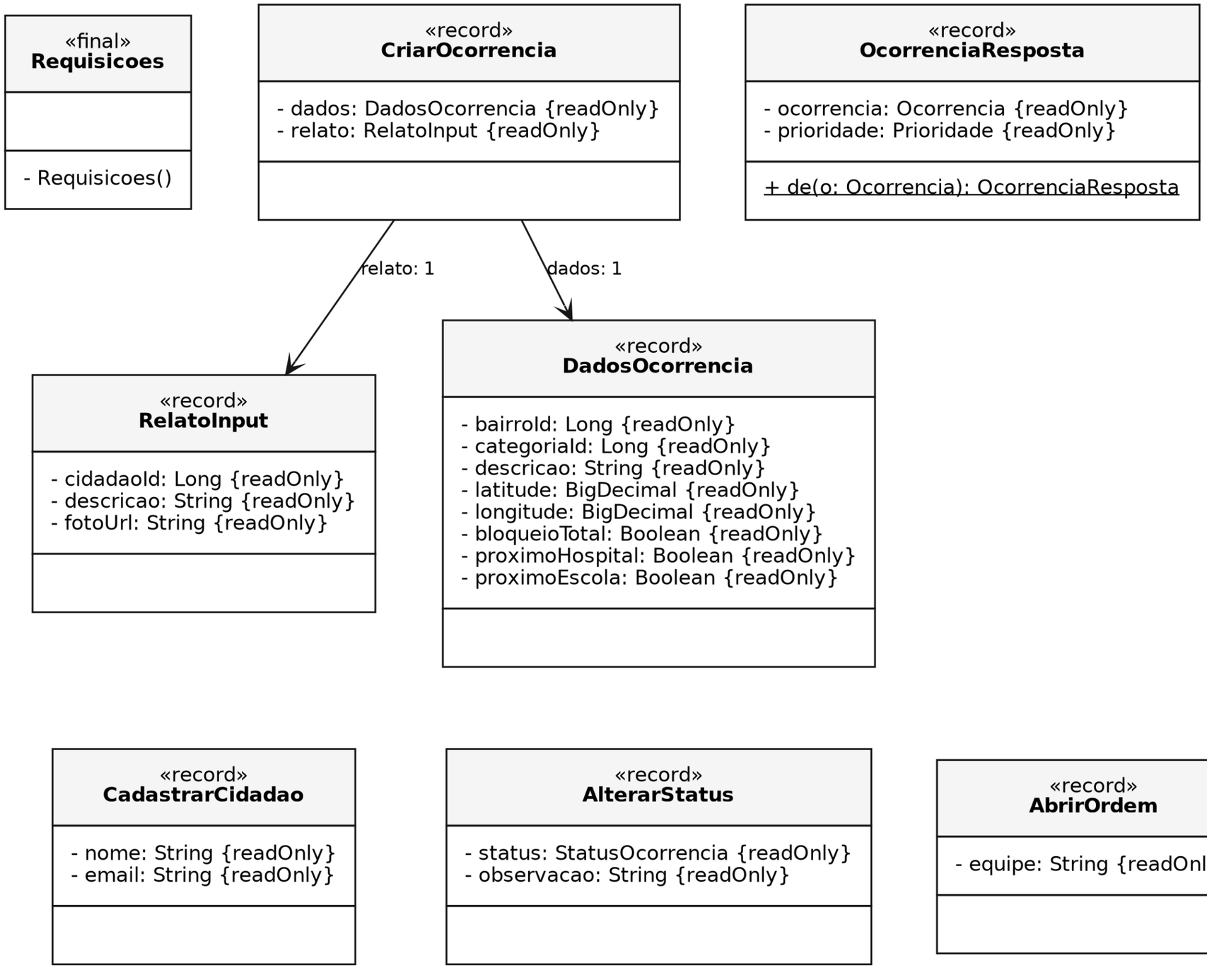
## Controllers da API

EventoPublico é um record aninhado em OcorrenciaController. As BOs são referências detalhadas na vista anterior. Visibilidade: + público, - privado, # protegido e ~ pacote. Membros estáticos sublinhados. Nos records, acessores e construtores implícitos foram omitidos.



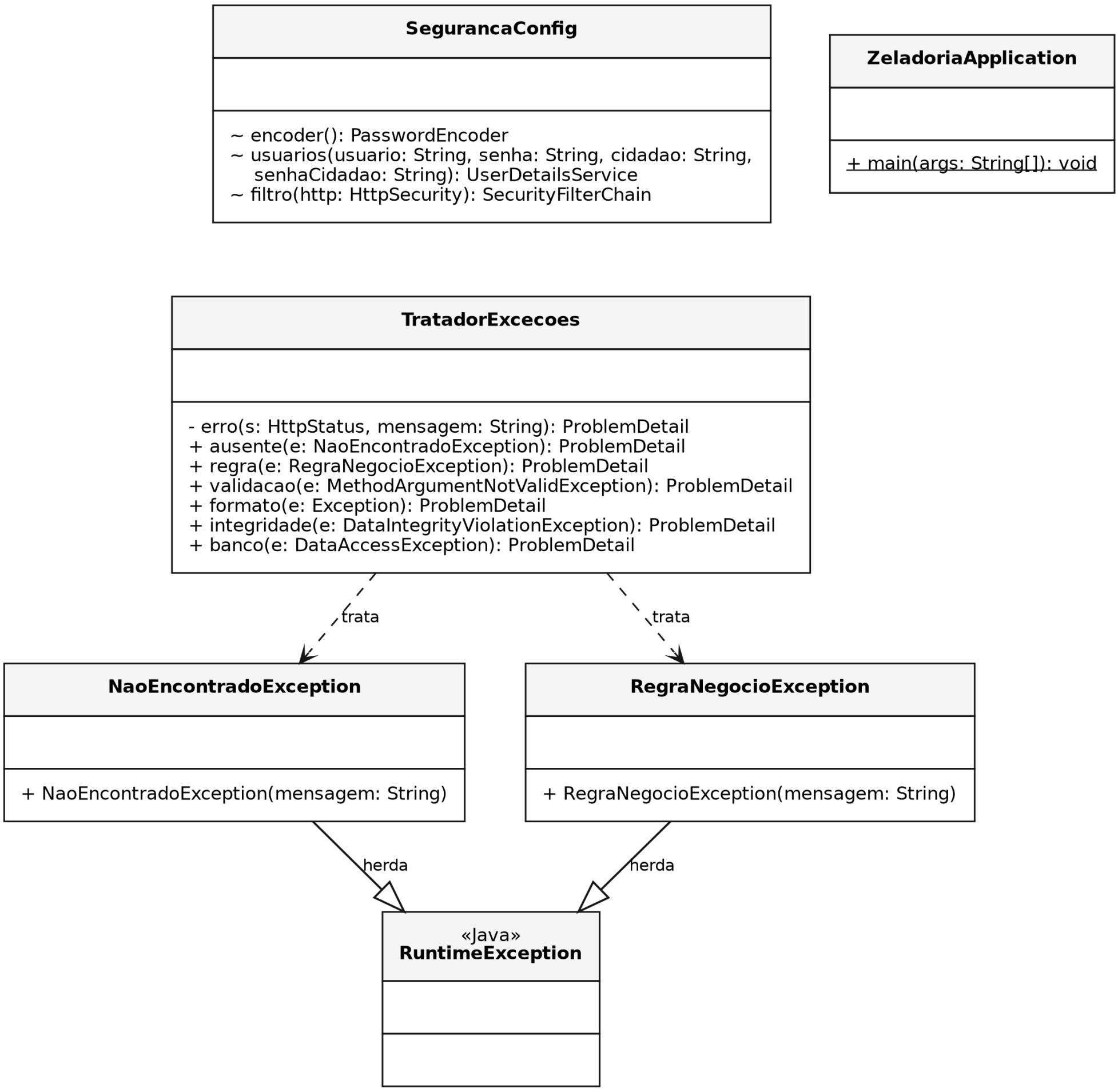
## Contratos de entrada e saída

Os seis records de entrada são aninhados em Requisicoes. OcorrenciaResposta é independente. Visibilidade: + público, - privado, # protegido e ~ pacote. Membros estáticos sublinhados. Nos records, acessores e construtores implícitos foram omitidos.



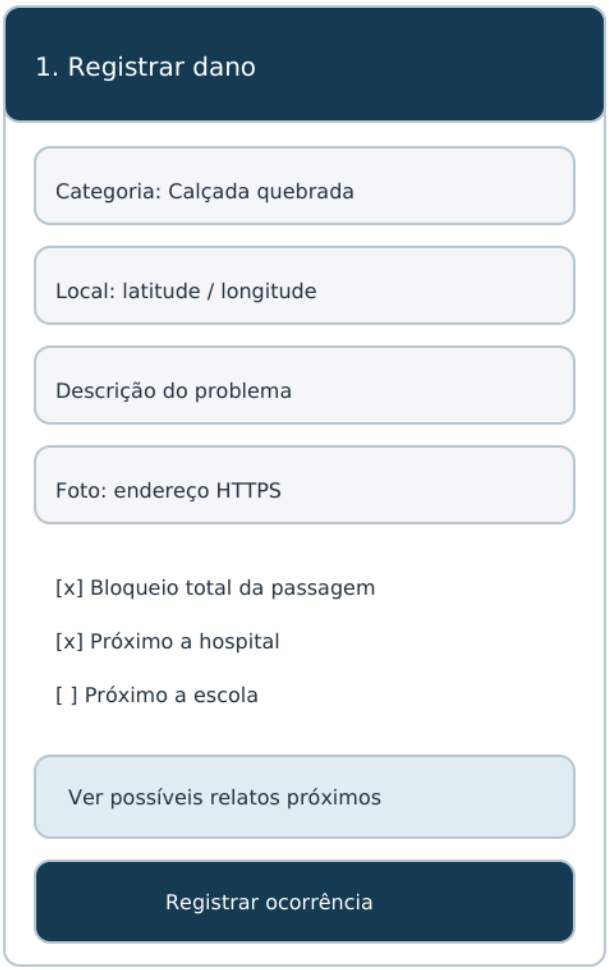
## Configuração e exceções

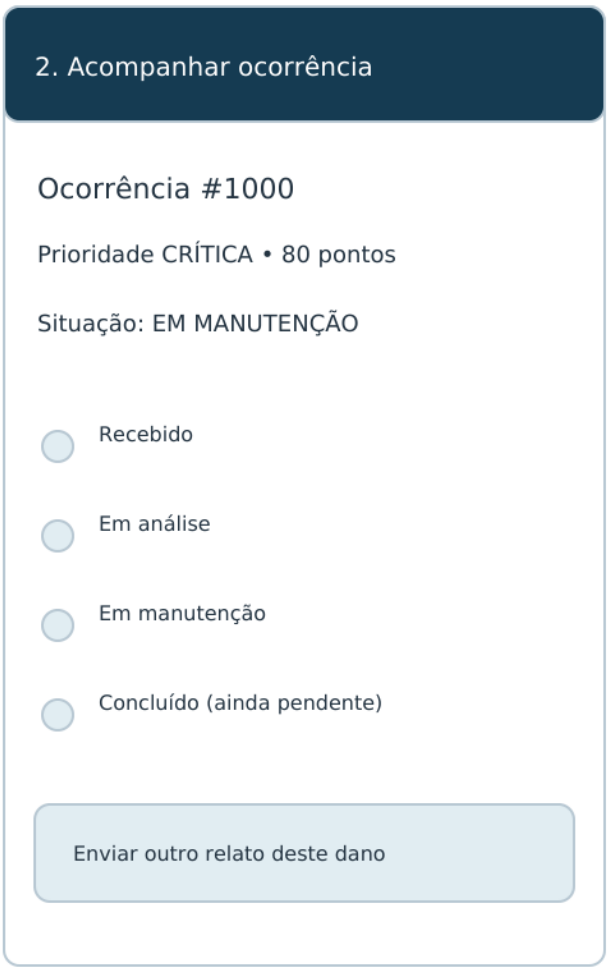
Triângulo vazado indica herança. RuntimeException pertence ao Java. Setas abertas indicam referências e dependências, preservando a semântica UML. Visibilidade: + público, - privado, # protegido e ~ pacote. Membros estáticos sublinhados. Nos records, acessores e construtores implícitos foram omitidos.



# 4 Protótipos de tela

Esboços para alinhar a integração com o responsável pelo Front-end. Não são capturas de uma interface implementada. O formulário propõe rótulos claros e estados também apresentados em texto.




## Painel de gestão

| Ocorrência e bairro | Prioridade | Estado e ação |
|---|---|---|
| 1000 \| Bela Vista | CRITICA \| 80 | EM_ANALISE \| abrir OS |
| 1001 \| Pinheiros | ALTA \| 50 | RECEBIDO \| analisar |

O painel pode agrupar por bairroId e permitir a próxima transição válida. Não há filtro de bairro implementado na API. Consulta de relatos, edição e manutenção exigem gestor. Fotos reais e upload dependem da integração.

# 5 Contrato HTTP

## Consultas públicas

| Método e rota | Perfil | Corpo e resposta |
|---|---|---|
| GET /api/bairros | Público | —<br>200; lista de bairros |
| GET /api/categorias | Público | —<br>200; categorias |
| GET /api/ocorrencias?limite=100 | Público | —<br>200; ocorrência + prioridade |
| GET /api/ocorrencias/{id} | Público | —<br>200 ou 404 |
| GET /api/ocorrencias/semelhantes | Público | —<br>200; candidatas até 30 m |
| GET /api/ocorrencias/{id}/historico | Público | —<br>200; status e data, sem autor |
| GET /actuator/health | Público | —<br>200 UP ou 503 DOWN |

Semelhantes recebe categoriaId, latitude e longitude como query parameters. Exemplo: categoriaId=1, latitude=-23.55 e longitude=-46.63.

## Cadastro e gestão

| Método e rota | Perfil | Corpo e resposta |
|---|---|---|
| POST /api/ocorrencias | Cidadão/gestor | CriarOcorrencia: dados + relato<br>201 + Location |
| PUT /api/ocorrencias/{id} | Gestor | DadosOcorrencia<br>200; edição antes de manutenção |
| PUT /api/ocorrencias/{id}/status | Gestor | status + observacao<br>200; fluxo validado |
| DELETE /api/ocorrencias/{id} | Gestor | —<br>204; arquivamento lógico |
| POST /api/ocorrencias/{id}/relatos | Cidadão/gestor | cidadaoId + descricao + fotoUrl<br>201; id do relato |
| POST /api/ocorrencias/{id}/ordens-servico | Gestor | equipe<br>201; id da OS |
| POST /api/cidadaos | Gestor | nome + email<br>201; id, não cria login |
| GET /api/gestao/ocorrencias/{id}/relatos | Gestor | —<br>200; relatos e fotos |
| GET /api/gestao/ocorrencias/{id}/ordens-servico | Gestor | —<br>200; OS e datas |

## Erros e contratos

| Código | Condição |
|---|---|
| 400 | JSON, enum, parâmetros ou campos inválidos. |
| 401 e 403 | Credenciais ausentes ou inválidas, ou perfil sem permissão. |
| 404 | Ocorrência ativa ou referência não encontrada. |
| 409 | Transição inválida, OS duplicada, identidade do relato incompatível ou restrição de integridade. |
| 503 | Falha de persistência ou banco indisponível. |

OcorrenciaResposta contém ocorrencia e prioridade. O histórico público retorna somente estado e data, sem autor. Foto é uma URL HTTPS; não existe envio de arquivo binário. Exemplos completos em examples/roteiro.http e CitizenTech_Zeladoria.postman_collection.json.

# 6 Execução e configuração

## Preparar o banco e a aplicação

1. Em schema vazio, executar os scripts DDL e DML no Oracle FIAP. Se o schema já estiver criado e carregado, não repetir esses scripts.

2. Abrir o pom.xml no IntelliJ como projeto Maven e selecionar JDK 21.

3. Configurar as variáveis abaixo em Run → Edit Configurations → Environment variables.

| Variável | Valor a configurar |
|---|---|
| DB_URL | URL JDBC correspondente à conexão FIAP. |
| DB_USER | Usuário Oracle FIAP. |
| DB_PASSWORD | Senha Oracle FIAP. |
| ADMIN_USER | Login do gestor, escolhido pelo grupo. |
| ADMIN_PASSWORD | Senha do gestor com pelo menos 12 caracteres. |
| CITIZEN_USER | Opcional; email de cidadão cadastrado no Oracle. |
| CITIZEN_PASSWORD | Opcional; senha da conta cidadã com pelo menos 12 caracteres. |
| PORT | Opcional; 8080 por padrão. Na execução local informada, foi usada a porta 8081. No Render, usar a variável fornecida pelo serviço. |

Exemplo de URL com service name: jdbc:oracle:thin:@//HOST:PORTA/SERVICE_NAME. Para SID, utilizar o formato correspondente informado pela FIAP. Credenciais do gestor são independentes das credenciais do banco.

## Executar e testar

Na raiz, executar mvn clean verify e mvn spring-boot:run, ou iniciar ZeladoriaApplication pelo IntelliJ com as variáveis configuradas. Confirmar a mensagem Started ZeladoriaApplication. Na configuração local atual, consultar http://localhost:8081/actuator/health e http://localhost:8081/api/ocorrencias.

A rota raiz não tem página inicial. Um 404 em http://localhost:8081/ não impede o funcionamento da API. No Postman, configurar baseUrl e credenciais do gestor; executar as requisições em ordem. IDs são capturados após a criação.

## 7 Diagrama de entidades e relacionamentos

Esquema de sete tabelas correspondente aos modelos e ao SQL. Substituir pela exportação final do Oracle Data Modeler. Relatos e histórico têm mínimo 0 no banco; a criação pela API garante o primeiro filho na transação.

